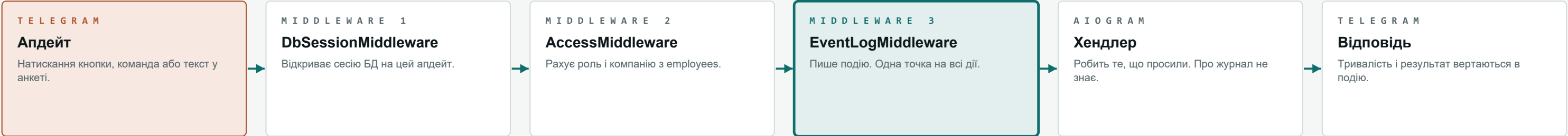


Журнал подій

Мета — бачити, куди люди тиснуть, де спотикаються і що в них не виходить. Ключове рішення: події пише middleware, а не хендлери. Через нього проходить кожен апдейт, тож логування неможливо забути при додаванні нового екрана — рівно так само, як edited_by проставляє репозиторій, а не той, хто пише редактор.

ТРАКТ ОДНОГО АПДЕЙТА



Запис події не має права зламати дію. Помилка журналу гаситься й іде в лог процесу, а не до користувача, і коміт події окремий від коміту дії — інакше збій запису статистики відкотив би створений рейс.

ЩО MIDDLEWARE БАЧИТЬ САМ

хто	tg_id, employee_id, company_id – з Access, який уже порахований
що	команда, callback_data або факт текстового вводу
де	FSM-стан на момент події: TripForm.ttn, TripEdit.value
коли	мітка часу, ISO-8601 UTC
скільки	тривалість хендлера, мілісекунди
чим скінчилось	ok або клас винятку
чи не двічі	update_id: Telegram повторює доставку при збої мережі

ЧОГО НЕ ПИШЕМО НІКОЛИ

Ніколи не пишемо ЗНАЧЕННЯ, які ввела людина: текст повідомлення, ПІБ, номери телефонів, номер ТТН, держномери. Пишемо ім'я поля й результат перевірки — цього досить, щоб побачити, де люди спотикаються, і це не перетворює журнал на другу копію персональних даних. payload — білий список ключів (trip_id, company_id, field, reason), а не «все, що є».

ЩО ДОВОДИТЬСЯ ПИСАТИ ЯВНО

denied	Відмова через права. Хендлер відповідає штатно, збоку це не видно — тому пишемо самі.
invalid	Ввід не пройшов перевірку: поле й причина. Значення — ні.
flow	Віхи анкети: form_started, form_submitted, form_abandoned.
dead_end	Вихід у меню одразу після відкриття екрана.

ПОРЯДОК РОБІТ

01 Таблиця й middleware плюс вимикач у .env	02 Явні події denied, invalid, flow	03 Екран «Активність» головному адміну, у боті	04 Ретенція й агрегати сирі події живуть обмежено
--	--	---	--

Таблиця events

Один рядок — одна дія однієї людини. Сесію не зберігаємо колонкою: «візит» — це події одного tg_id з проміжком менше 30 хвилин, і рахується вона запитом. Так поріг можна змінити заднім числом, не переписуючи вже накопичене.

СТРУКТУРА

events		
id	INTEGER	PK
ts	TEXT NOT NULL	
update_id	INTEGER	UQ
tg_id	BIGINT	
employee_id	INTEGER	FK
company_id	INTEGER	FK
kind	TEXT NOT NULL	
name	TEXT NOT NULL	
state	TEXT	
payload	TEXT	
duration_ms	INTEGER	
ok	INTEGER NOT NULL	
error	TEXT	

KIND

command	/start, /help, /cancel
callback	натискання інлайн-кнопки
message	текстовий ввід у стані анкети
denied	відмовлено через права
invalid	ввід не пройшов перевірку
flow	віха анкети: почав, надіслав, кинув
error	виняток у хендлері

ІНДЕКСИ

(ts)	вибірка за період
(tg_id, ts)	шлях однієї людини
(kind, name, ts)	топи й воронки

НА ЯКІ ПИТАННЯ ВІДПОВІДАЄ

- 01

Воронка рейсу
скільки почали анкету, дійшли до кроку N, підтвердили
- 02

Де кидають
останній стан перед мовчанням понад 15 хвилин
- 03

Які поля дратують
kind=invalid, згруповано за полем — топ за тиждень
- 04

Куди тиснуться без прав
kind=denied: або модель прав крива, або кнопка веде не туди
- 05

Глухі кути
екрани, з яких одразу виходять у меню
- 06

Повільні місця
p95 duration_ms за name
- 07

Хто користується
активні люди за тиждень, у розрізі компаній

ДЕ ЦЕ ЧИТАТИ

- у боті

«Активність» головному адміну: 7 днів, топ дій, відмови, незавершені анкети
- через API

GET /api/admin/events за X-Admin-Token — той самий секрет, що й для заявок
- руками

SQL по events: сесія рахується запитом, а не зберігається колонкою

РЕТЕНЦІЯ, ОБСЯГ І МЕЖИ

tg_id — персональні дані, тому сирі події не живуть вічно: пропозиція — 90 днів, далі лишаються добові агрегати без прив'язки до людини. Обсяг невеликий: приблизно 10-30 подій на рейс, тобто близько 1500 на день при 50 рейсах — SQLite тримає це спокійно. Але писати в неї буде і бот, і журнал, а writer у SQLite один: коли записи почнуть конкурувати, журнал переїжджає в окремий файл events.db, і лише потім — у Postgres.